



Programação Avançada 2025-26

[14] Padrões de Software | MVP

Patrícia Macedo

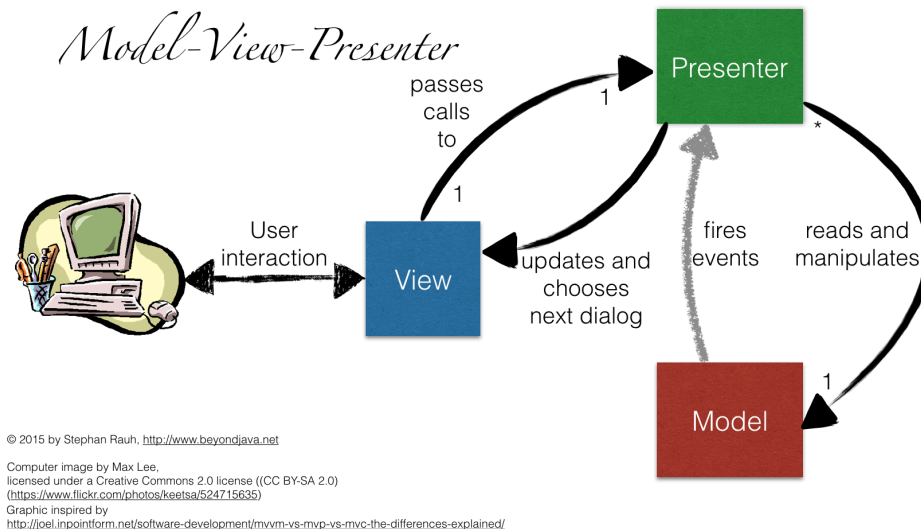
Enquadramento (1/2)

Evolução dos padrões arquiteturais para interfaces

- O crescimento das interfaces gráficas (GUIs) nas décadas de 90 e 2000 trouxe maior complexidade na gestão de eventos, estado e lógica de apresentação.
- O padrão **MVC**, criado nos anos 70 no Xerox PARC para Smalltalk, começou a mostrar limitações em ambientes com componentes visuais ricos.
- Tornou-se necessário um padrão que **separasse melhor a interface da lógica** e permitisse **testes unitários** com dependências da UI

MVP

- O **MVP** é um padrão de arquitetura e define 3 entidades: O Model, o View e o Presenter.
- Este modelo tem como finalidade de separar a camada de apresentação das camadas de dados e regras de negócio.



MVP | Model

- Sabe tudo sobre:
 - Os dados persistentes que devem ser apresentados;
 - As operações que serão aplicadas para transformar os objectos.
- Nada sabe sobre:
 - As interfaces do utilizador;
 - Como os dados serão mostrados;
 - As acções das interfaces usadas para manipular os dados.

MVP | View

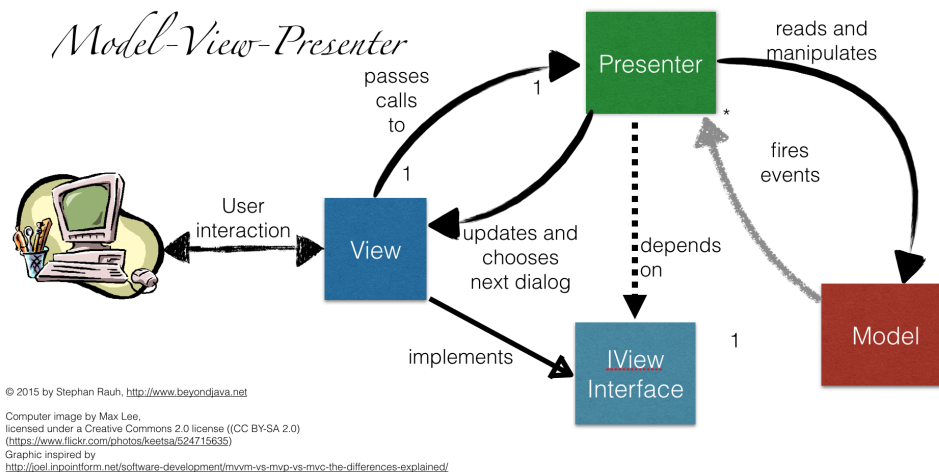
- Refere-se ao objecto Presenter;
- Define como os dados serão visualizados pelo utilizador;
- Recebe as notificações do Presenter, para atualizar a visualização dos dados.

MVP | Presenter

- Sincroniza as acções do View com as acções realizadas pelo Model;
- Trabalha com a informação recebida pela View, processa e envia o pedido ao Model;
- É notificado pelo Model, das alterações realizadas, e em função disso atualiza a View.

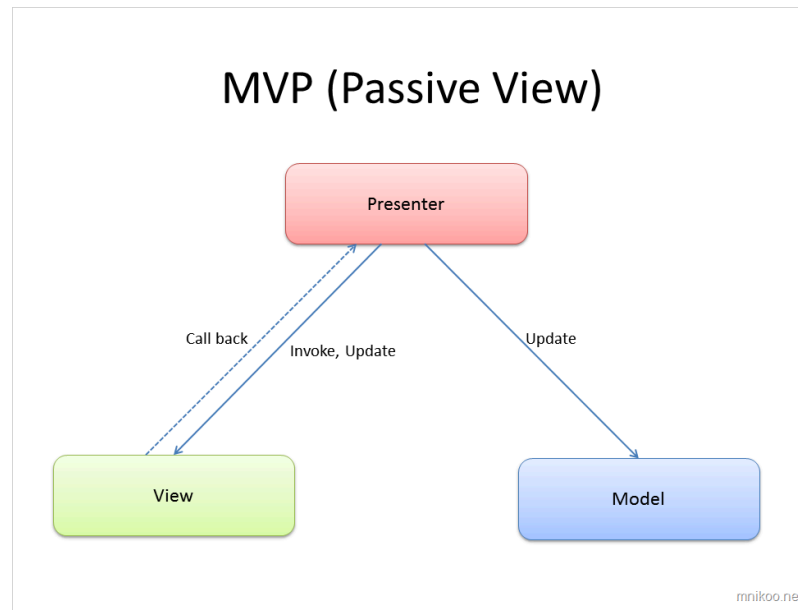
Boas Práticas de Implementação

- Para tornarmos a classe que implementa o Presenter da View, devemos antes definir uma interface IView e tornar a classe Presenter dependente da Interface, para desta forma ser mais fácil substituirmos a View.



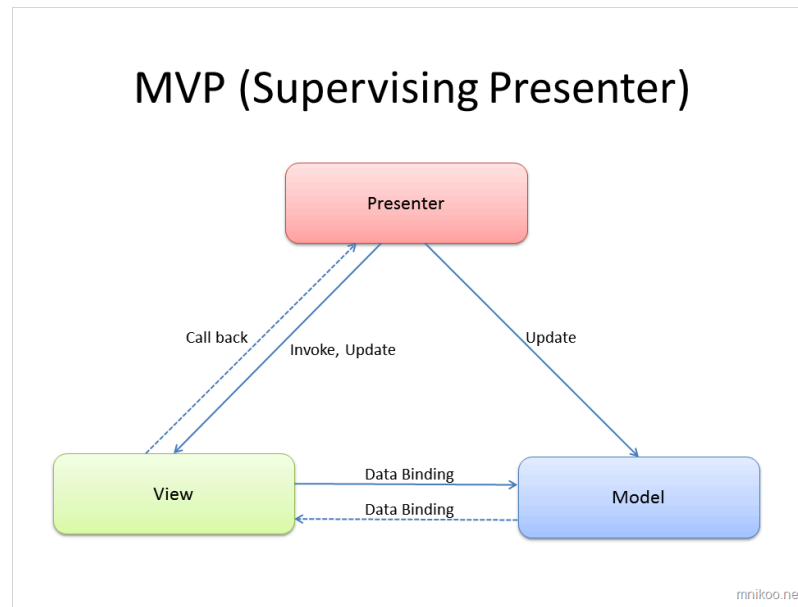
MVP | Passive View

- O Modelo de MVP apresentado é conhecido por Passive View, já que a View tem um papel passivo e apenas é atualizada em função de ordem do Present.

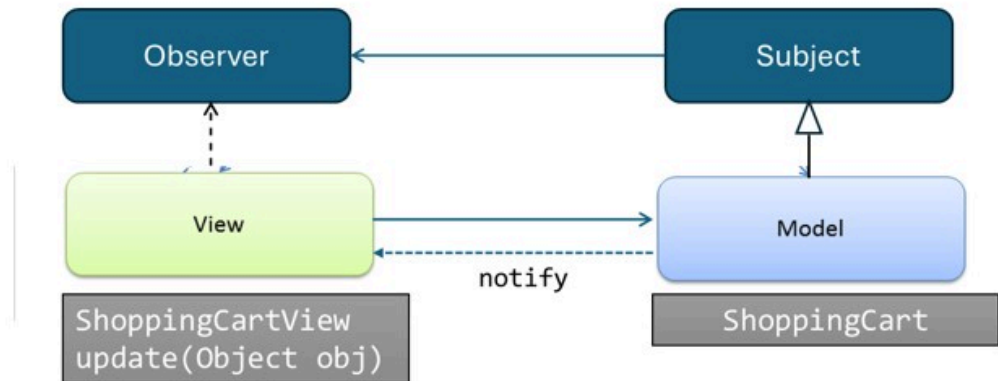
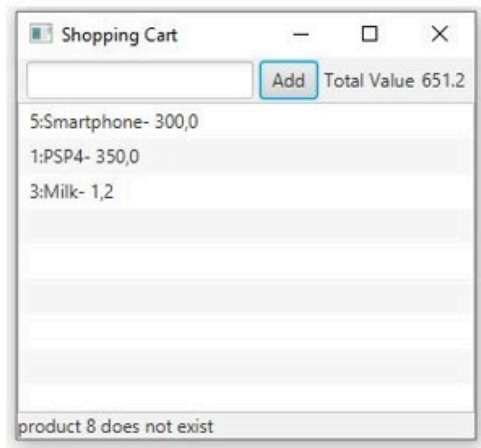


MVP | Supervising Presenter

- No modelo MVP- Supervising Presenter, a View interage diretamente com o modelo através do mecanismo de data-binding. - O presenter atualiza o modelo , e manipula o estado da View em situações mais complexas, onde é necessário introduzir lógica na apresentação dos components.



Revisitando o Model-View



Usando o padrão Observer, implementamos o padrão Model-View para separar a Interface do Utilizador com o modelo de negócio.

- O `ShoppingCart` assume o papel de Model, e notifica a View (`ShoppingCartView`) cada vez que o Carrinho de compras é alterado.

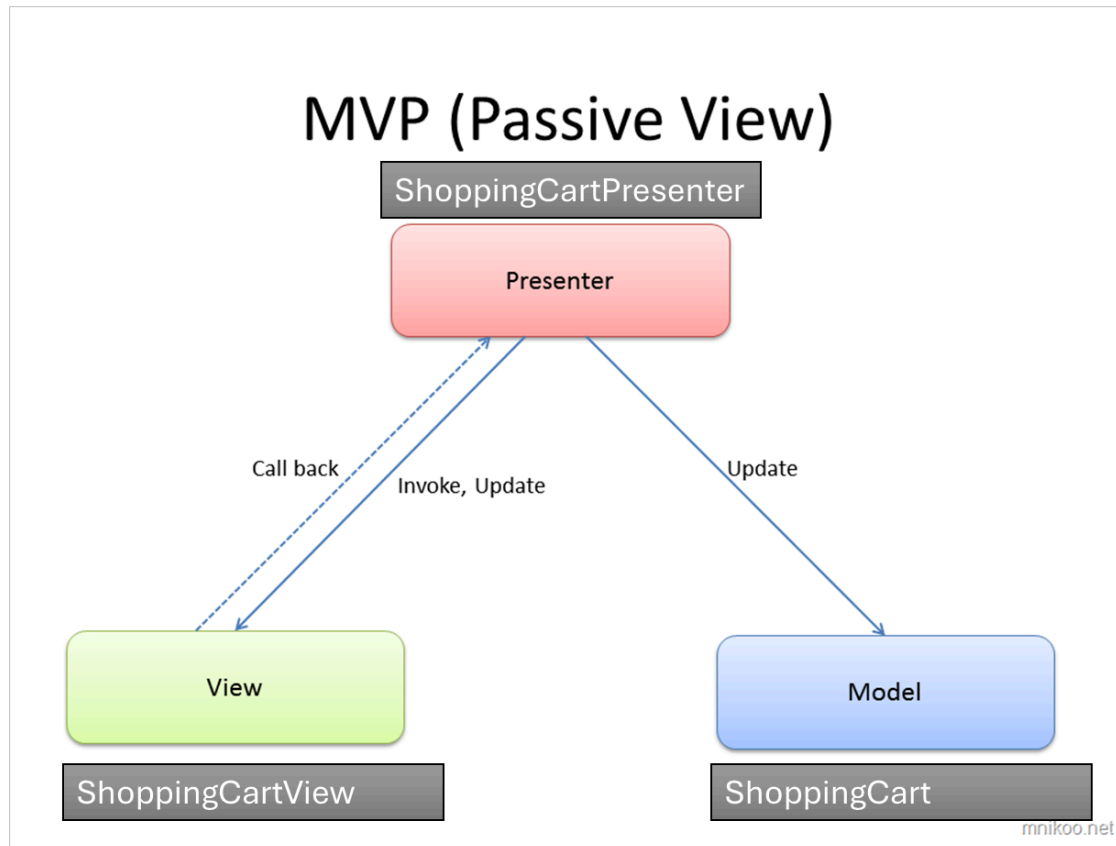
Desafio 1 -MVP -Passive View

- Implementar a app da gestão do carrinho de compras usando o padrão MVP, em vez do padrão Model-View.

NOTA - Aceda ao código base aqui:

https://github.com/estsetubal-pa-geral/MVP_ShoppingCart

Desafio 1:Modelação



Desafio 1 - Modelação

- **Model:** A classe `ShoppingCart` é responsável por gerir a lista de produtos que compoe o carrinho de compras.
- **View:** A classe `ShoppingCartUI` implementa em JavaFX a interface grafica do ShoppingCart.
- **Presenter:** A classe `ShoppingCartPresenter` é responsável por sincronizar as acções do View (pressionar os botões) com as acções realizadas pelo Model. Esta sincronização é realizada pelo método `doAddProduct`, através dos métodos `doAdd`. É reponsável por solicitar à View para mostrar mensagens de alerta, caso a operação de adicionar Produto gere uma excepção.

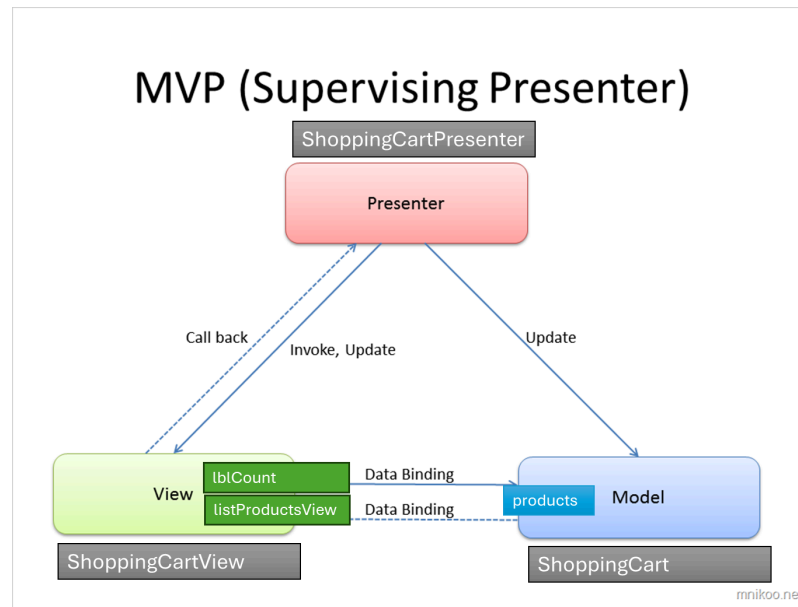
Desafio 1: Exercícios

1. Coloque um botão para remover produtos. E adicione o comportamento.
 2. Adicione um alarme, para quando o valor total do carrinho exceder um determinado valor.
- Nesse caso, não deve ser possível adicionar mais produtos ao carrinho e o total deve ficar a vermelho.

Desafio 2: MVP Supervioning Presenter

A variante **MVP Supervising Presenter**. permite que a View assuma alguma responsabilidade, especialmente no que toca à lógica simples de interface.

- O **Presenter** continua a ser responsável pela lógica de alto nível.
- A **View** usa data binding directamente com o modelo e pode actualizar componentes visuais sem ter de pedir ao Presenter para cada pequena alteração.



Desafio 2: Uso do observableList

Aqui utilizamos data binding implícito do JavaFX, ligando automaticamente a `ListView` à `ObservableList<Product>` do modelo. Deste modo, sempre que um produto é adicionado ou removido, a lista na interface é atualizada sem código extra.

```
public class ShoppingCart {  
    private String name;  
    private ObservableList<Product> products; // utilizada para permitir data binding com a View
```

```
public class ShoppingCartUI {  
    private ListView<Product> listProductsView;  
    private void initComponents() {  
        // ...  
        this.listProductsView = new ListView<>(model.getProducts()); // binding implícito  
    }  
}
```


Desafio 2: databinding with lblCount

Neste caso vamos usar o DataBinding do JavaFX, estabelecendo uma ligação automática entre o componente `Label lblCount`, que apresenta o custo total do carrinho de compras, e o modelo através da sua lista observável de produtos.

Assim, sempre que um produto for adicionado ou removido da lista `ObservableList<Product>`, o valor total é recalculado automaticamente e o texto do Label é atualizado sem necessidade de código adicional na lógica da interface.

```
private void databindingTotalCost(Label lblCount) {
    lblCount.textProperty().bind(
        Bindings.createDoubleBinding(
            () -> (double) model.getTotal(), // call your method
            model.getProducts()               // dependency: the ObservableList
        ).asString("%.2f")                    // format as string with 2 decimal places
    );
}
```

Desafio 2: Exercícios

Adicione a seguinte funcionalidade usando DataBinding

- Introduza um novo label na interface gráfica que mostre o produto mais caro no carrinho.

Nota: Altere as classes que assume o papel de View e Model para implementar esta funcionalidade.

Padrão MVP — Principais Vantagens

- ◆ **Separação clara de responsabilidades**
 - Model (dados), View (interface) e Presenter (lógica) ficam independentes.
- ◆ **Maior testabilidade :**
 - Presenter pode ser testado com *unit tests* sem interface gráfica.
- ◆ **Manutenção e evolução facilitadas**
 - Alterações na View não afetam a lógica e vice-versa.

Padrão MVP — Principais Vantagens (cont)

♦ Reutilização de código - O mesmo Presenter pode servir várias Views (desktop, web, mobile).

♦ Menor acoplamento e maior coesão

- Comunicação através de interfaces → código mais limpo e robusto.

♦ Adequado para UIs complexas

- Mantém a View simples, enquanto o Presenter controla interações e estado.

◆ Onde o MVP é mais utilizado

- **Web:** GWT, Backbone.js, AngularJS (1.x)
- **Mobile (Android):** Moxxy, Mosby, Nucleus (padrão muito adotado antes do MVVM)
- **Desktop:** Java Swing e JavaFX (padrão recomendado, não imposto), WinForms

◆ Ideia-chave

- **MVP** não é normalmente obrigatório pela framework;
- é escolhido como padrão arquitetural para melhorar organização e testabilidade.

Saber mais

- Model-View-Presenter Pattern in Java: Enhancing UI Logic Separation for Cleaner Code - <https://stackoverflow.com/questions/30554694/mvp-javafx-and-components-references>
- Multiple Scenes JavaFX MVP <https://dev.to/r0land013/multiple-scenes-javafx-mvp-38i4>
- MVP (Model View Presenter) Architecture Pattern in Android <https://www.scaler.com/topics/android/mvp-architecture-android/>